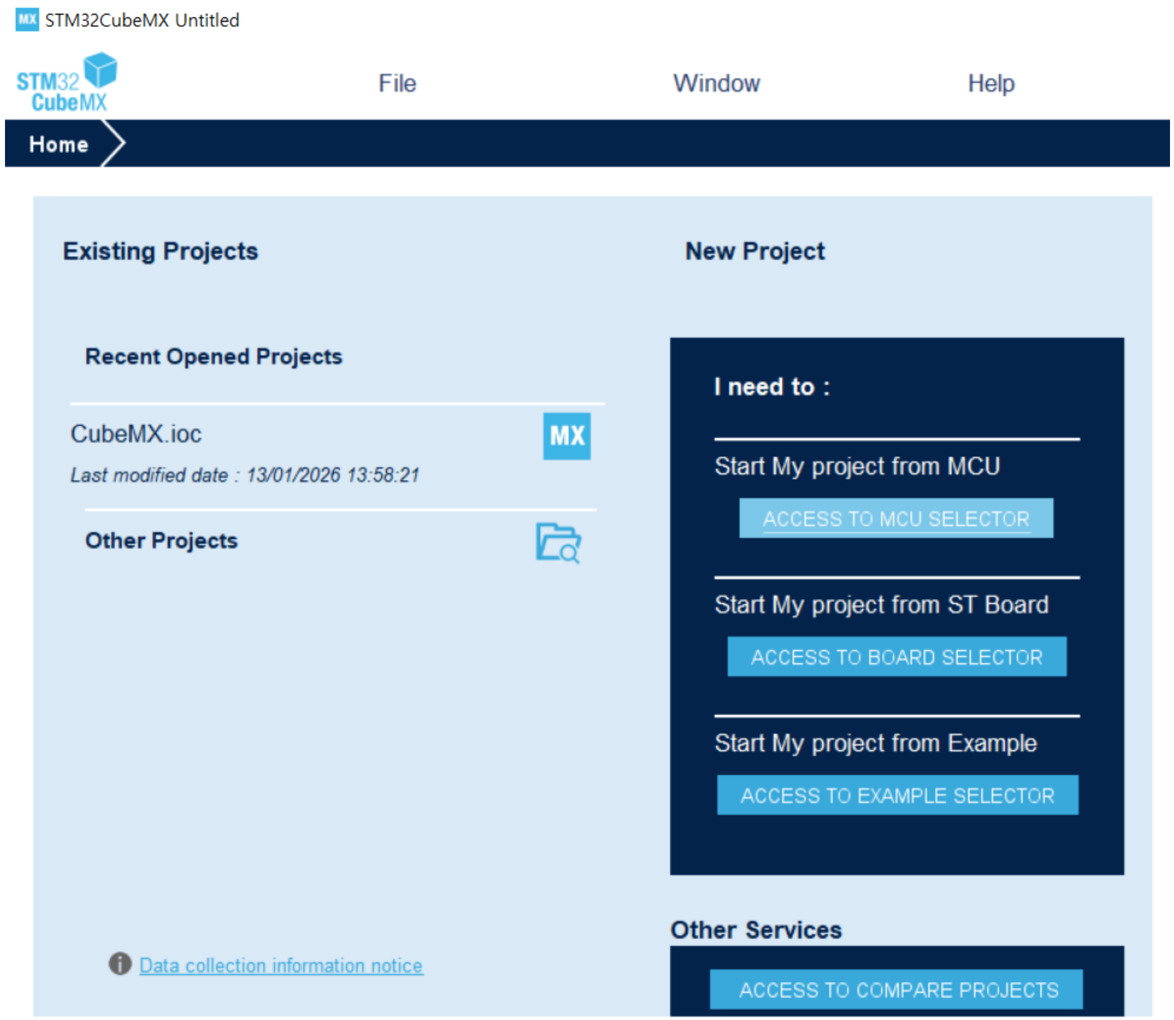
SHT20 – 습도 센서 실습

2026.06.17

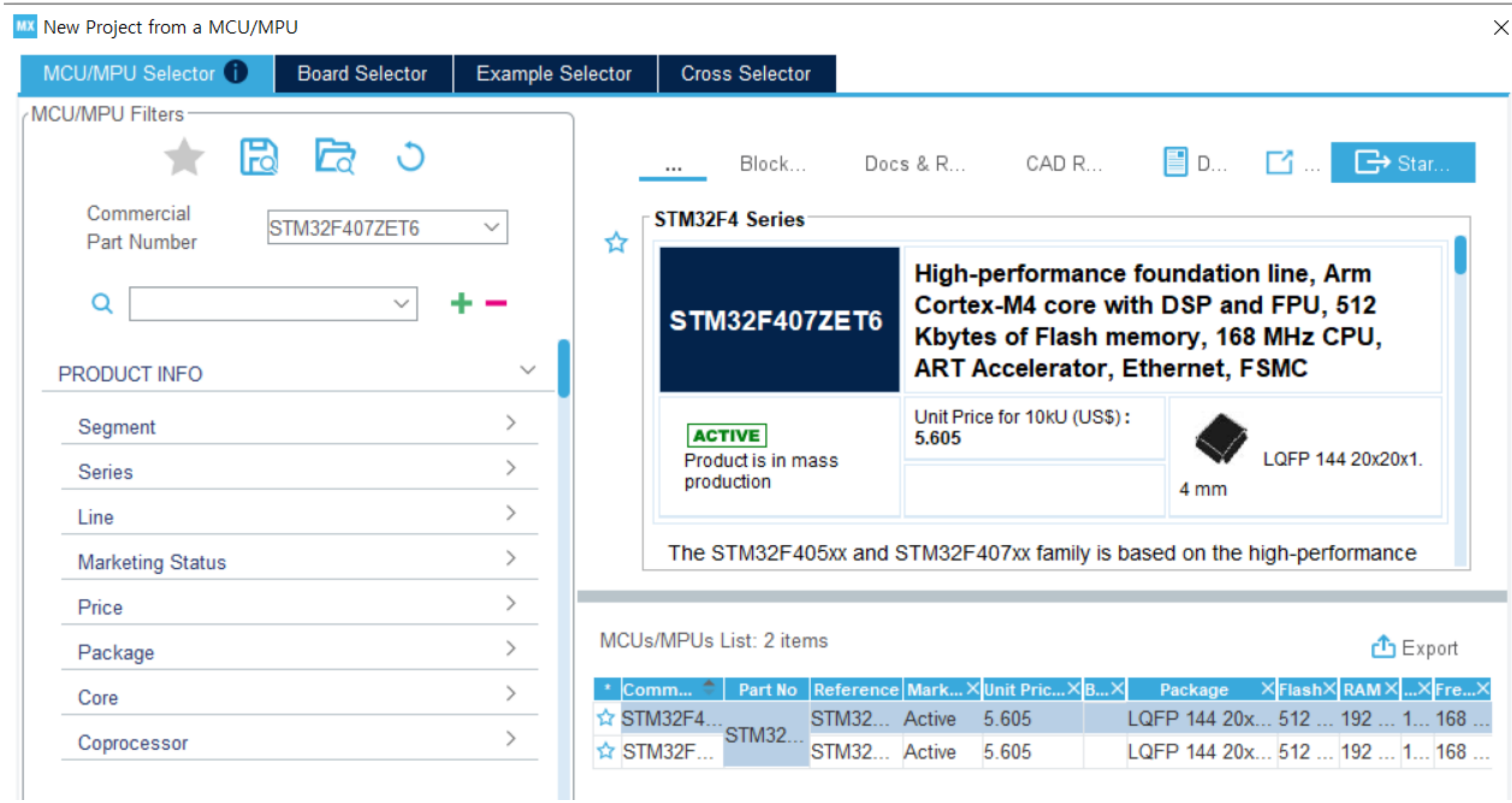
Project 생성

- STM32CubeMX 실행 후 "ACCESS TO MCU SELECTOR" 선택



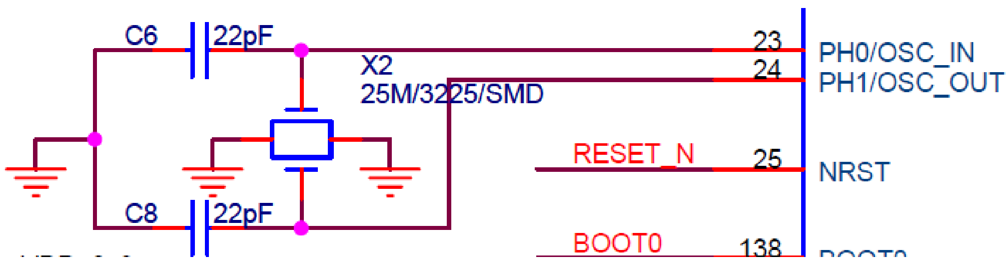
Project 생성

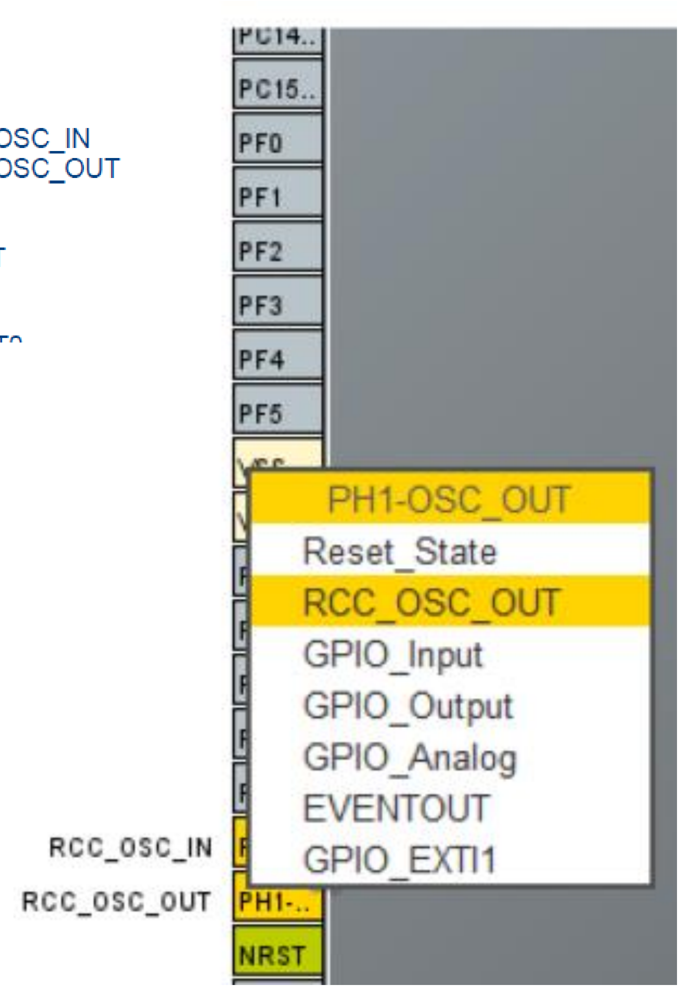
- STM32F407ZET6 선택 후 "Start Project” 클릭



Project 생성

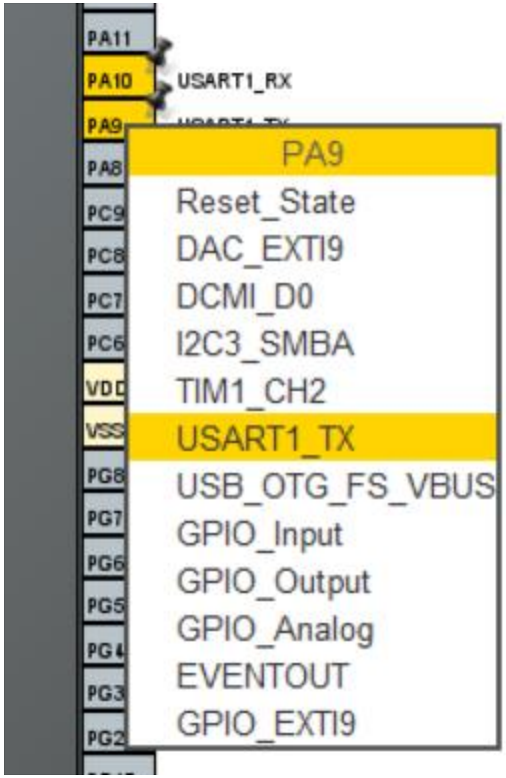
- HSE(25MHz) 핀 설정 – 회로도 참조
- HSE : PH0-OSC_IN, PH1-OSC_OUT





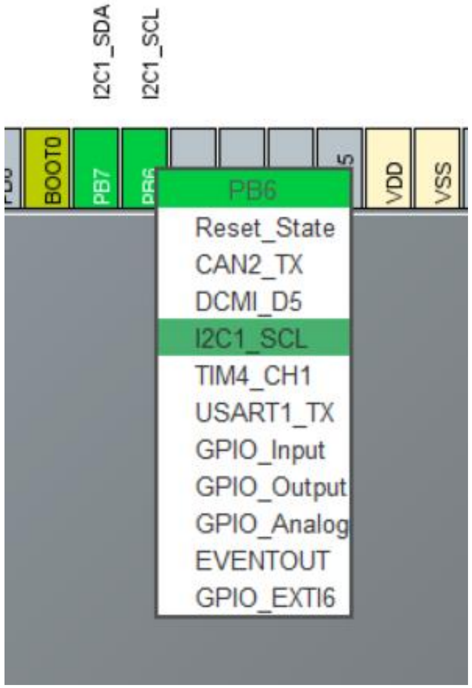
Project 생성

- USART1핀 설정 – 회로도 참조
- UART1 : PA9-USART1_TXD, PA10-USART1_RXD



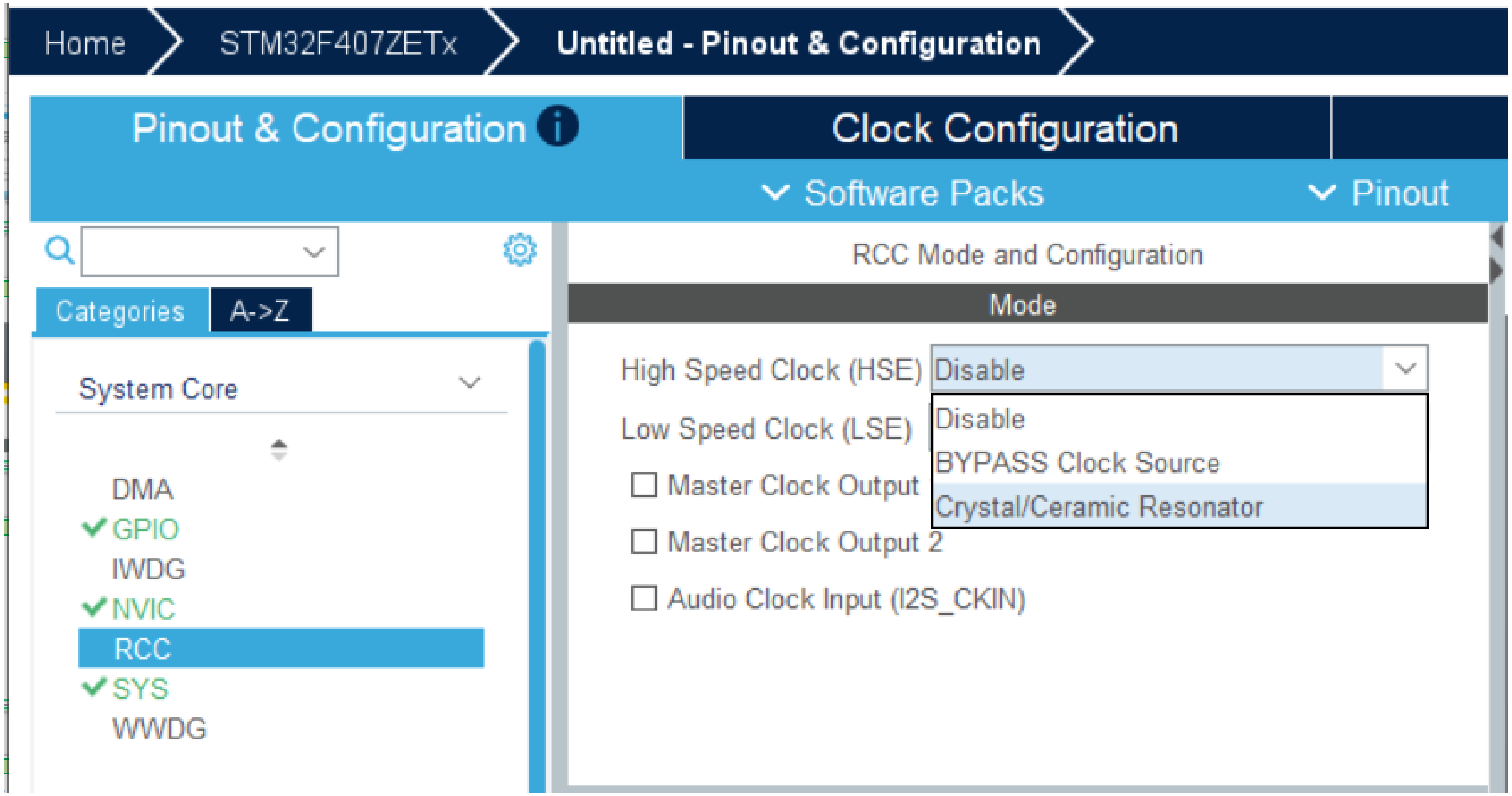
Project 생성

- I2C 핀 설정 – 회로도 참조
- I2C : PB6-I2C1_SCL, PB7-I2C1_SDA



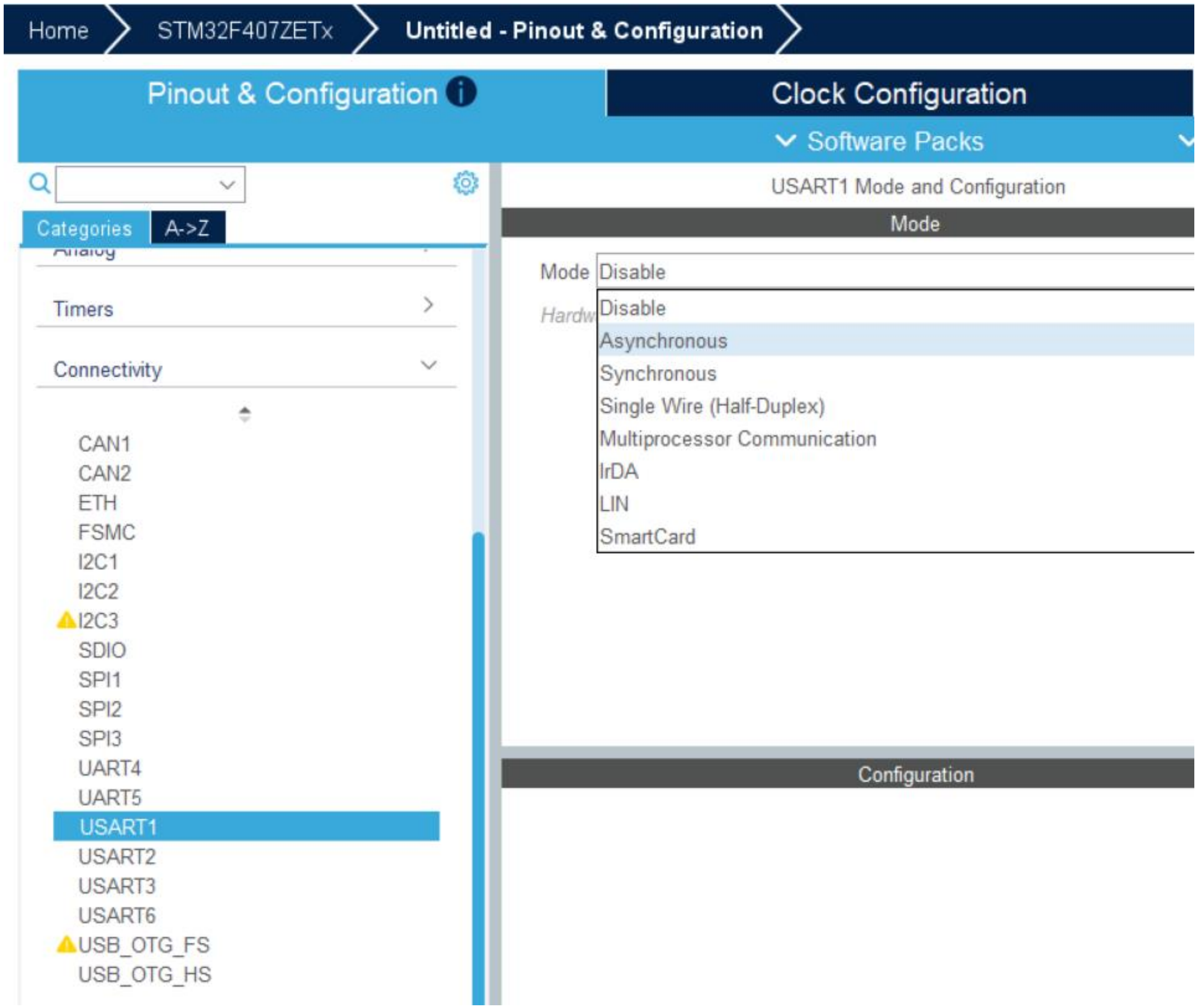
Project 생성

- System Core -> RCC에서 HSE 활성화 -> Crystal/Ceramic Resonator 선택



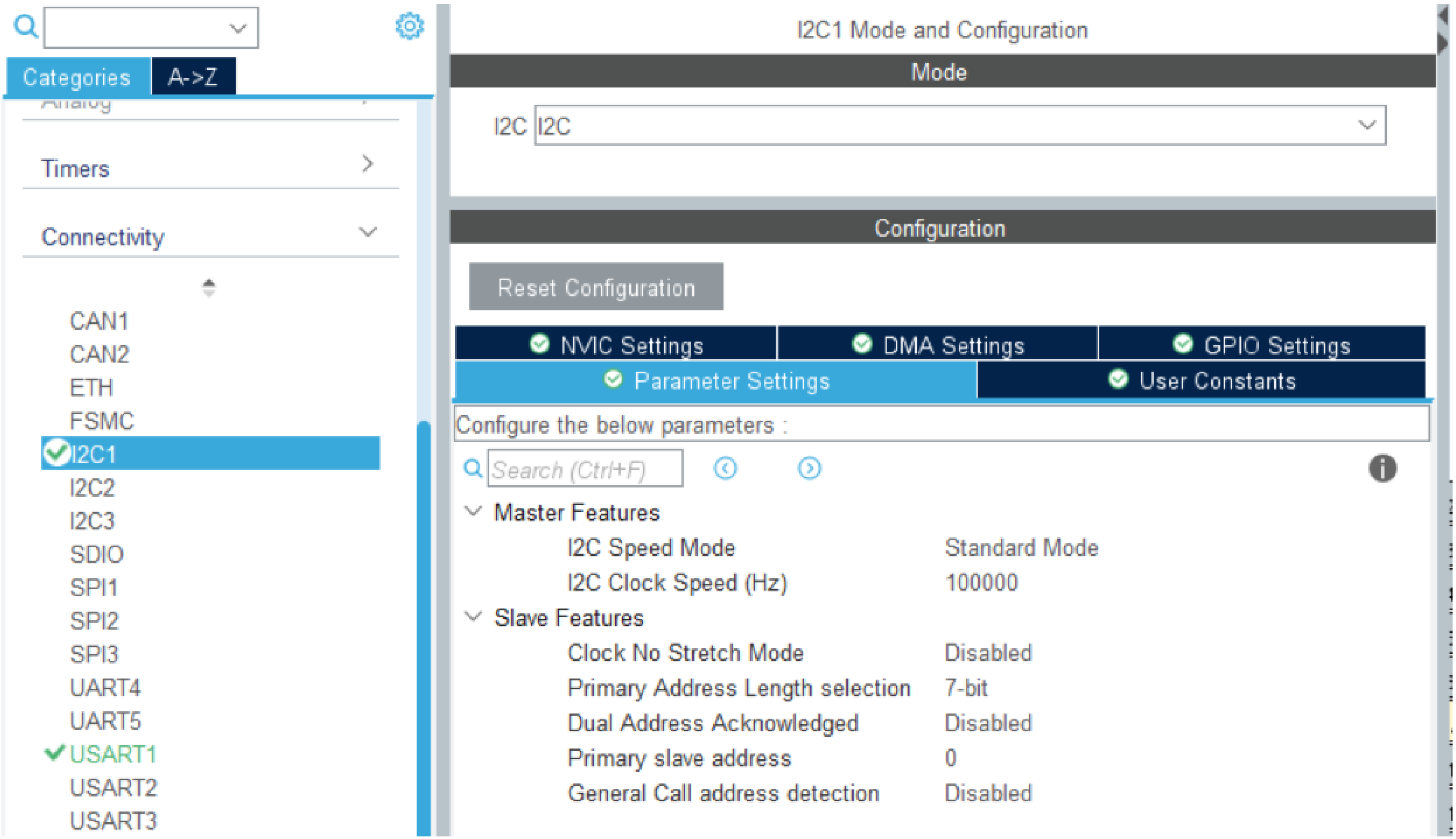
Project 생성

- Connectivity -> USART1 -> Asynchronous 선택



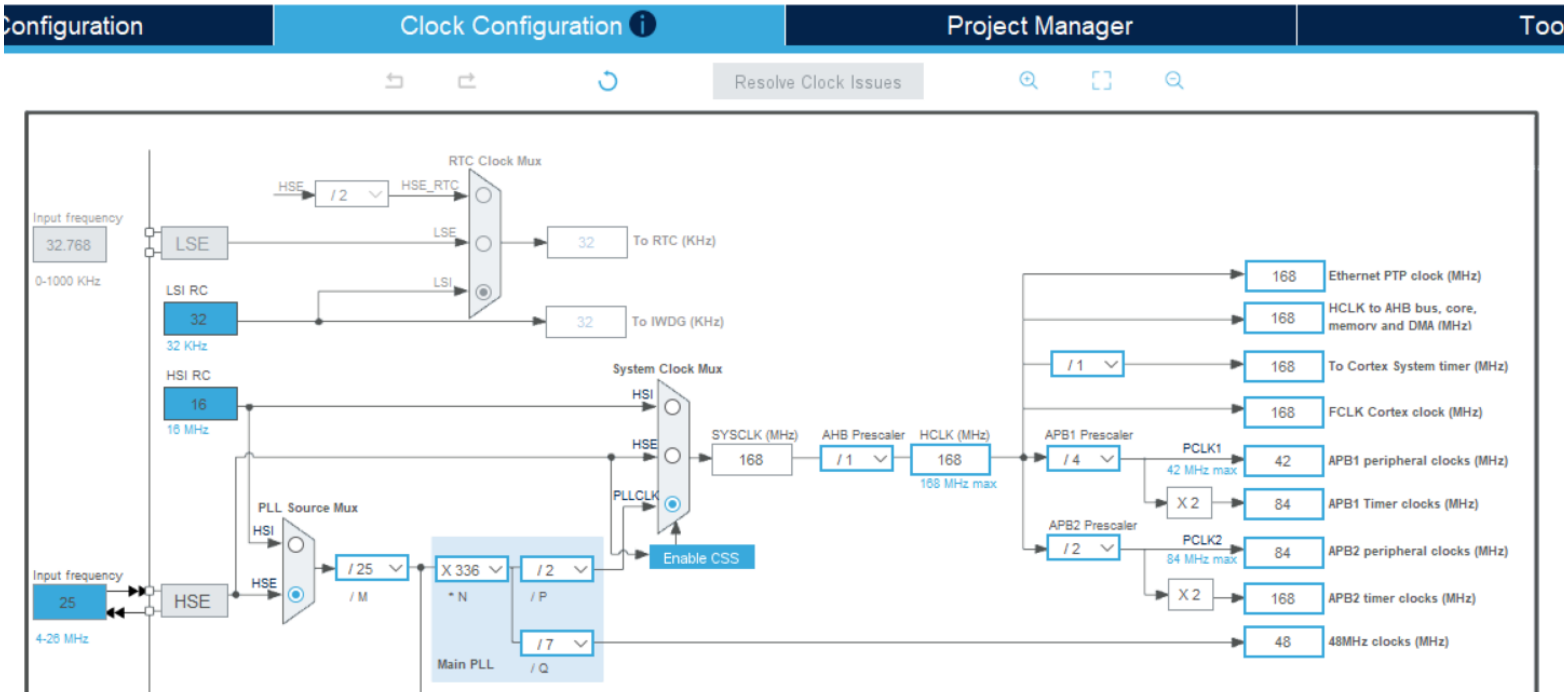
Project 생성

- Connectivity -> I2C1 -> Mode -> I2C 선택



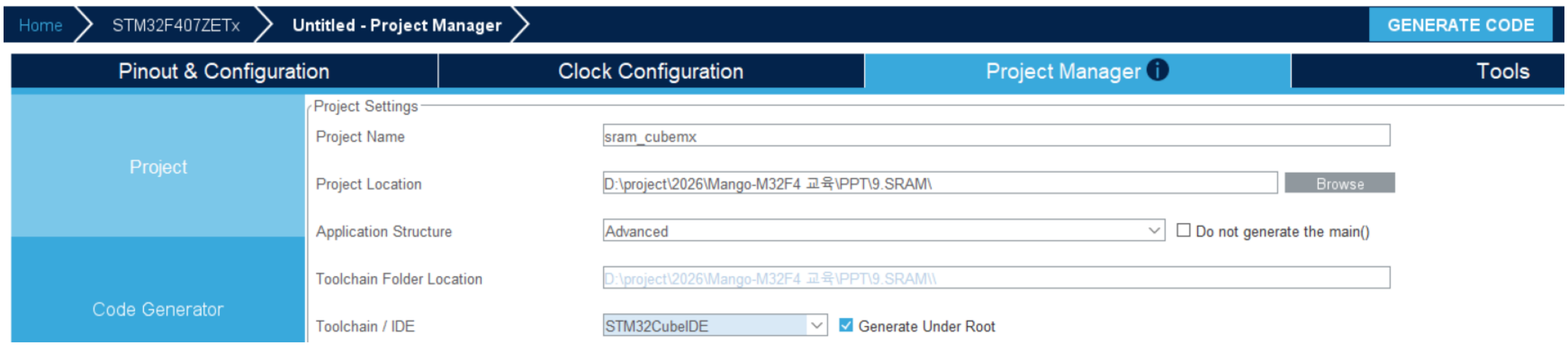
Project 생성

- Clock Configuration 클릭 -> Yes 클릭



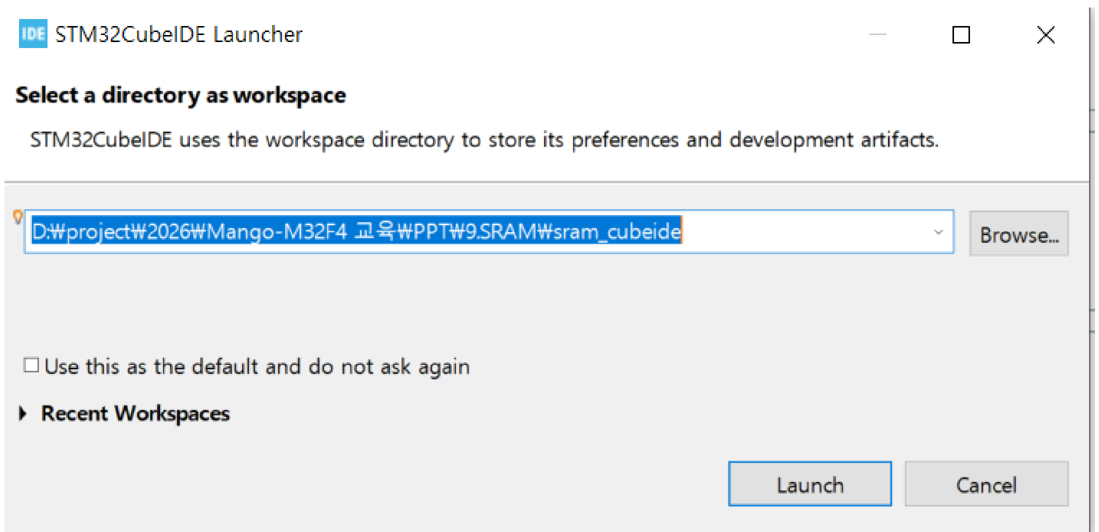
Project 생성

- Project Manager에서 Project Name 기입, IDE를 STM32CubeIDE로 선택 후 GENERATE CODE 클릭



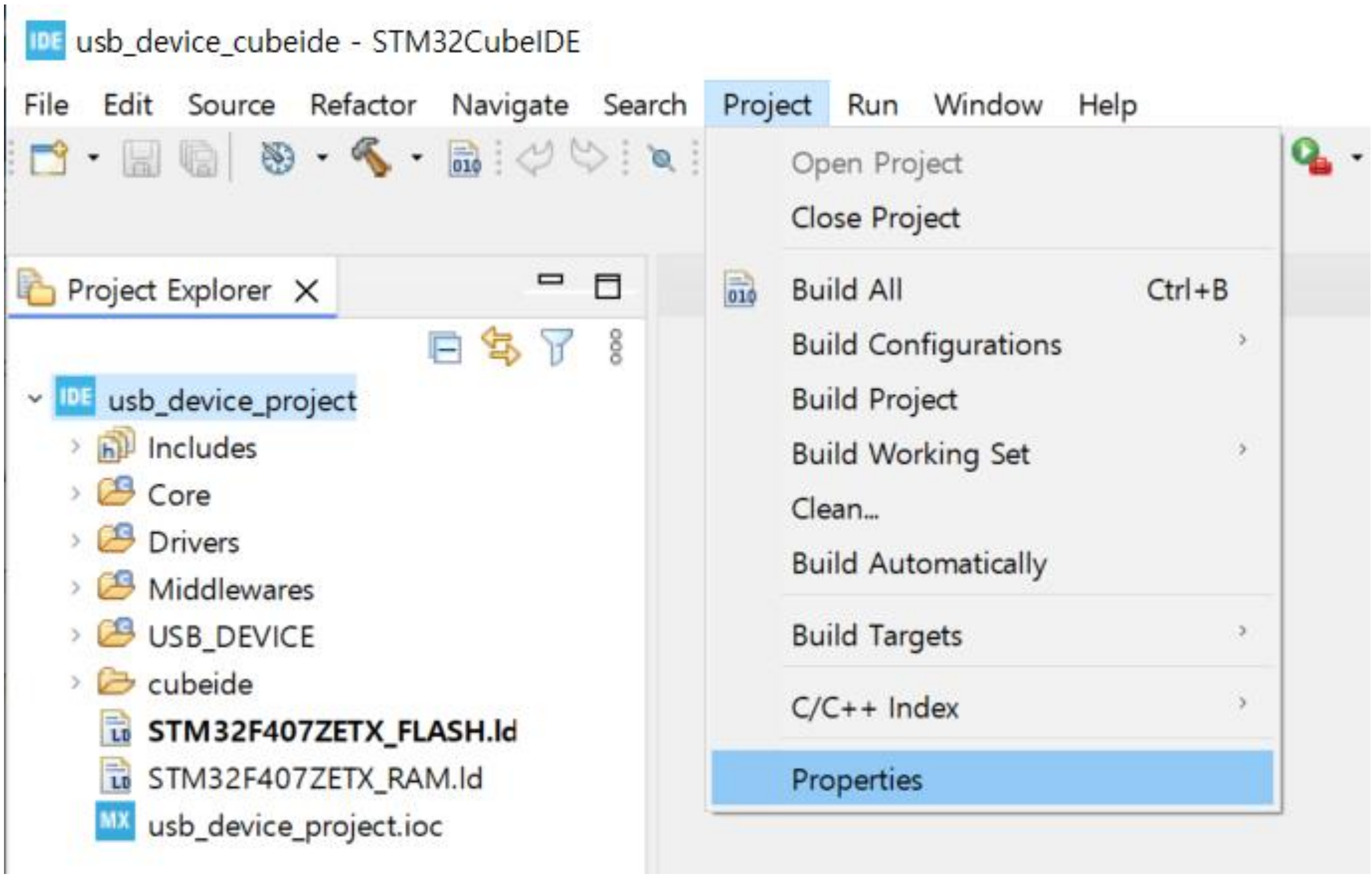
Project 생성

- Open Project 클릭하여 STM32CubeIDE 실행, workspace는 CubeMX와 다른 폴더로 지정



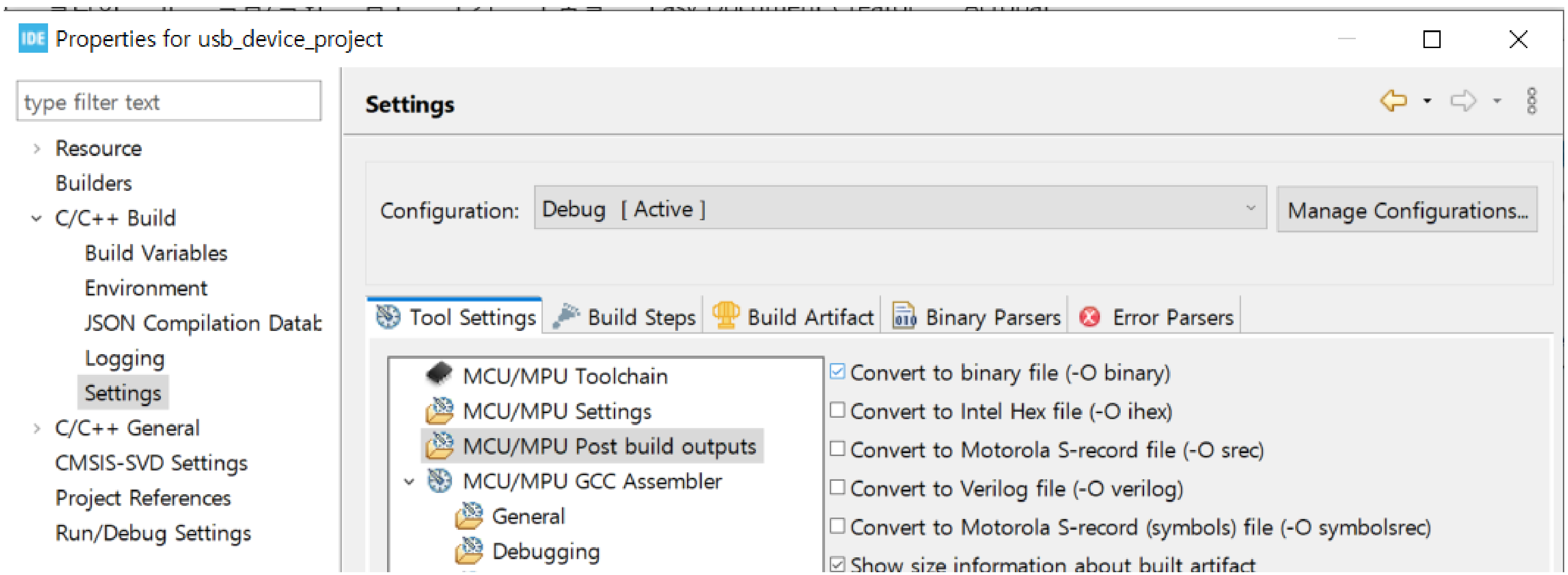
BIN 파일 생성 설정

- STM32CubeProgrammer로 시리얼 다운로드하기 위해서는 BIN파일이 필요.
- Project -> Properties 클릭



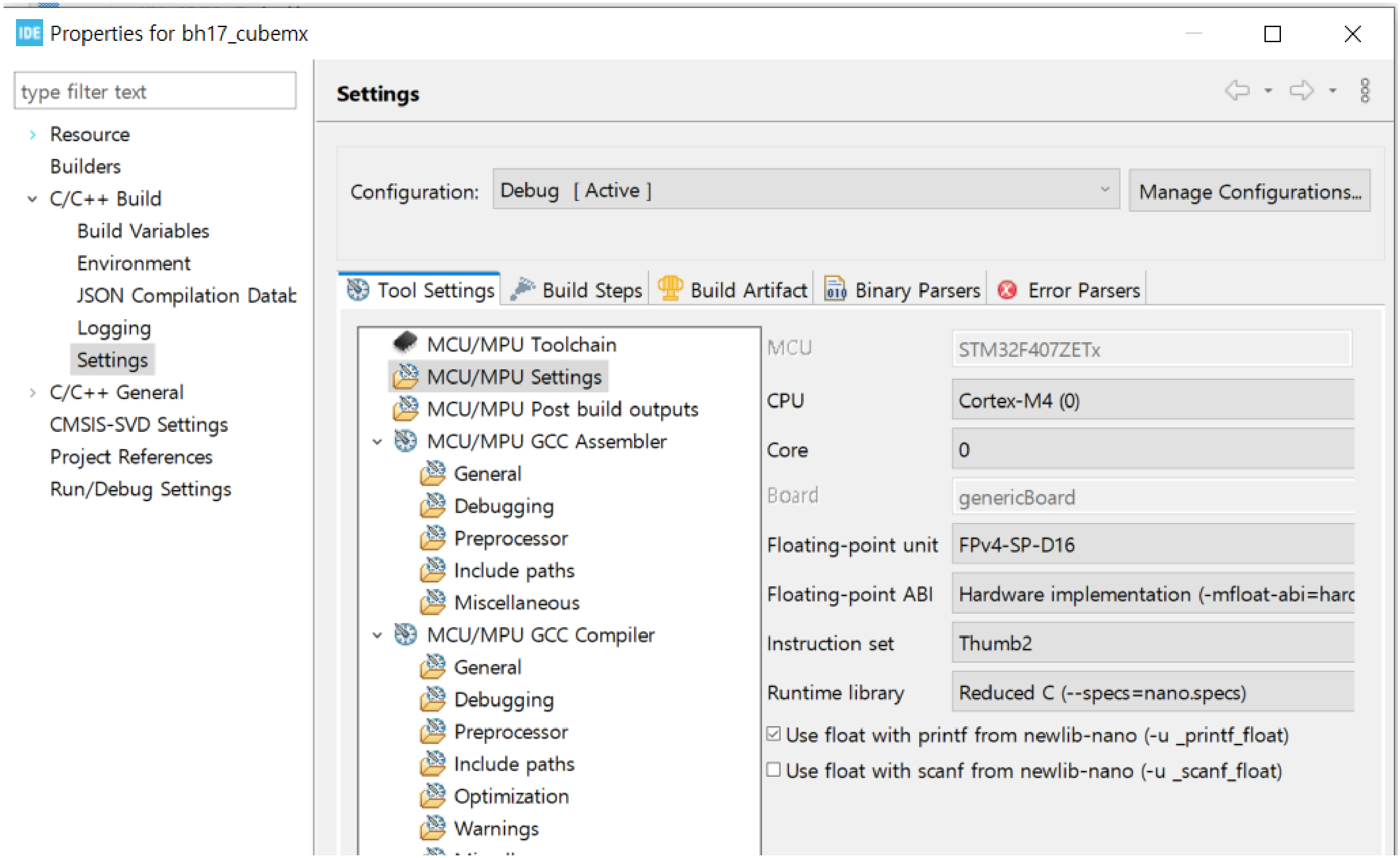
BIN 파일 생성 설정

- C/C++ Build -> Settings -> Tool Settings -> MCU/MPU Post build outputs => Convert to binary file(-O binary) 클릭.



빌드 설정

- C/C++ Build -> Settings -> Tool Settings -> MCU/MPU Settings => Use float with printf from newlib-nano(-u _printf_float) 클릭.



SHT20 데이터시트

- Slave Address : 0x40

After sending the Start condition, the subsequent I²C header consists of the 7-bit I²C device address '1000'000' and an SDA direction bit (Read R: '1', Write W: '0'). The

- Command Table

Command	Comment	Code
Trigger T measurement	hold master	1110'0011
Trigger RH measurement	hold master	1110'0101
Trigger T measurement	no hold master	1111'0011
Trigger RH measurement	no hold master	1111'0101
Write user register		1110'0110
Read user register		1110'0111
Soft reset		1111'1110

Table 6 Basic command set, RH stands for relative humidity, and T stands for temperature

- Humidity Conversion

- **Relative Humidity conversion**

With the relative humidity signal output S_{RH}, the relative humidity is obtained by the following formula (result in %RH), no matter which resolution is chosen:

$$RH = -6 + 125 \times \frac{S_{RH}}{2^{16}}$$

- Temperature Conversion

- **Temperature conversion**

The temperature T is calculated by inserting temperature signal output S_{Temp} into the following formula (result in °C), no matter which resolution is chosen:

$$Temp = -46.85 + 175.72 \times \frac{S_{Temp}}{2^{16}}$$

Code 수정

- UART1으로 로그를 내 보내기 위해 아래 코드 추가 : core/src/main.c

```
/* Private user code ---------------------------------------------------------*/
/* USER CODE BEGIN 0 */
#define hUART huart1
int _write( int32_t file , uint8_t *ptr , int32_t len )
{
   /* Implement your write code here, this is used by puts and printf for example
*/
   for ( int16_t i = 0 ; i < len ; ++i )
   {
      HAL_UART_Transmit( &hUART, ptr++, 1, 100);
   }
   return len;
}
/* USER CODE END 0 */


/**
* @brief The application entry point.
* @retval int
*/
int main(void)
{
```

Code 수정

- SHT20의 slave address는 0x40이다.

```
/* USER CODE BEGIN PD */

#define SHT20_I2C_ADDR (0x40 << 1)

#define REG_RTEMP_READ_HOLD          0xE3
#define REG_RHUM_READ_HOLD           0xE5
#define REG_SOFT_RESET               0xFE

//CRC error macros
#define CRC_MATCH            1
#define CRC_NOTMATCH         0

//CRC macros
#define CRC_POLYNOMIAL        0x131   // Generator Polynomial X^8 + X^5+ X^4+ 1
#define CRC_POLYNOMIAL_WIDTH    9
/* USER CODE END PD */
```

Code 수정

- CRC 함수.

```
static int crc_check(uint16_t dt, uint8_t crc)
{
    uint16_t polynomial = CRC_POLYNOMIAL << (16 - CRC_POLYNOMIAL_WIDTH);
    uint16_t result = dt;
    uint16_t top_bit = 0x8000;

    for(int i=0;i<16;i++)
    {
        if(result & top_bit) result ^= polynomial;
        result <<= 1;
    }

    result >>= 8;

    return crc == result ? CRC_MATCH : CRC_NOTMATCH;
}
/* USER CODE END 0 */
```

Code 수정

- 초기화시에 Software Reset를 한다.

```
int main(void)
{
    uint8_t reg;
    uint8_t data[3];
    uint16_t data_raw;
    uint8_t crc;
    int raw_value;
    HAL_StatusTypeDef status;

  printf("I2C-SHT20 test\r\n");

  HAL_Delay(1000);
  reg = REG_SOFT_RESET;
  status = HAL_I2C_Master_Transmit(&hi2c1, SHT20_I2C_ADDR, &reg, 1, 1000);
  HAL_Delay(300);
```

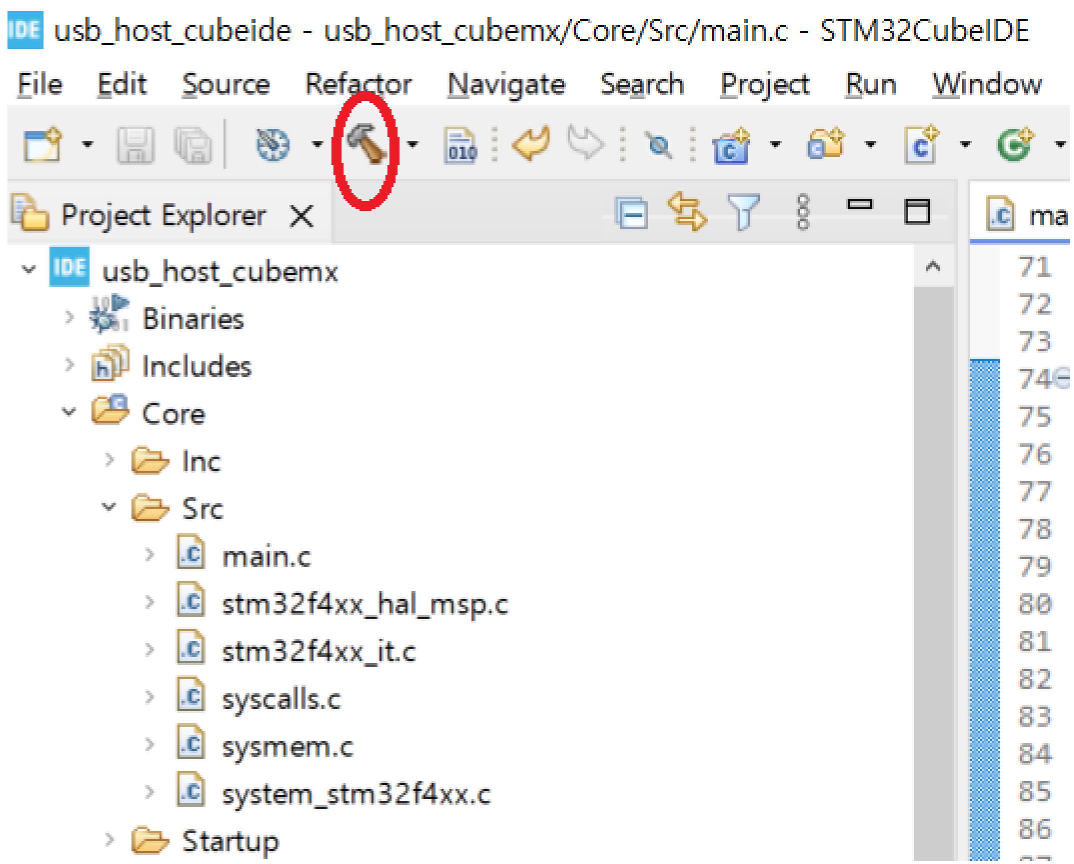
Code 수정

- 1초 주기로 온도와 습도 정보를 표시한다.

```
while (1)
 {
		HAL_Delay(1000);
			reg = REG_RTEMP_READ_HOLD;
			status = HAL_I2C_Master_Transmit(&hi2c1, SHT20_I2C_ADDR, &reg, 1, 1000);
			status = HAL_I2C_Master_Receive(&hi2c1, SHT20_I2C_ADDR, data, 3, 1000);
			data_raw = (data[0] << 8) | data[1];
			crc = data[2];
			if(crc_check(data_raw, crc) == CRC_MATCH)
			{
				data_raw &= 0xFFFC;
				raw_value = ((17572 * data_raw) >> 16) - 4685;

				printf("TEMP=%d.%d°C\r\n", raw_value / 100, raw_value % 100);
			}

			reg = REG_RHUM_READ_HOLD;
			status = HAL_I2C_Master_Transmit(&hi2c1, SHT20_I2C_ADDR, &reg, 1, 1000);
			status = HAL_I2C_Master_Receive(&hi2c1, SHT20_I2C_ADDR, data, 3, 1000);
			data_raw = (data[0] << 8) | data[1];
			crc = data[2];
			if(crc_check(data_raw, crc) == CRC_MATCH)
			{
				data_raw &= 0xFFFC;
				raw_value = ((125 * data_raw) >> 16) - 6;

				printf("HUMID=%d%%\r\n", raw_value);
			}
 }
}
```

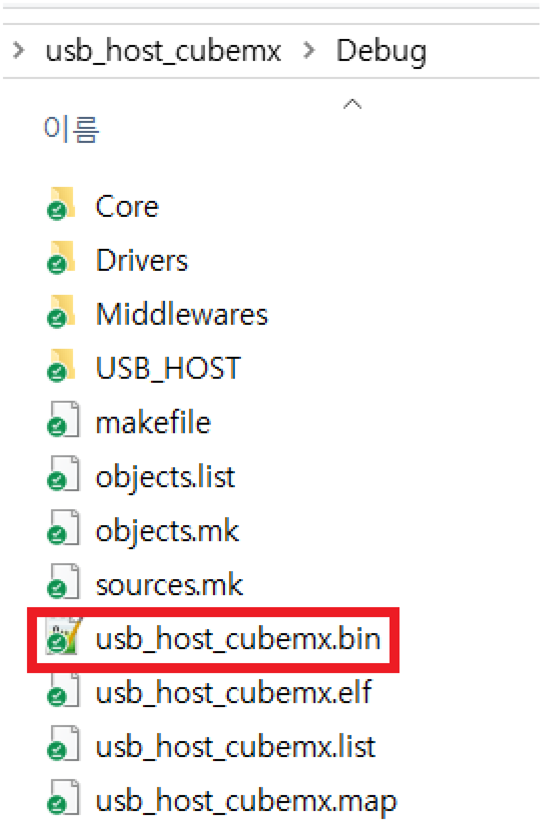
Build

- 아래 버튼 클릭하여 Build



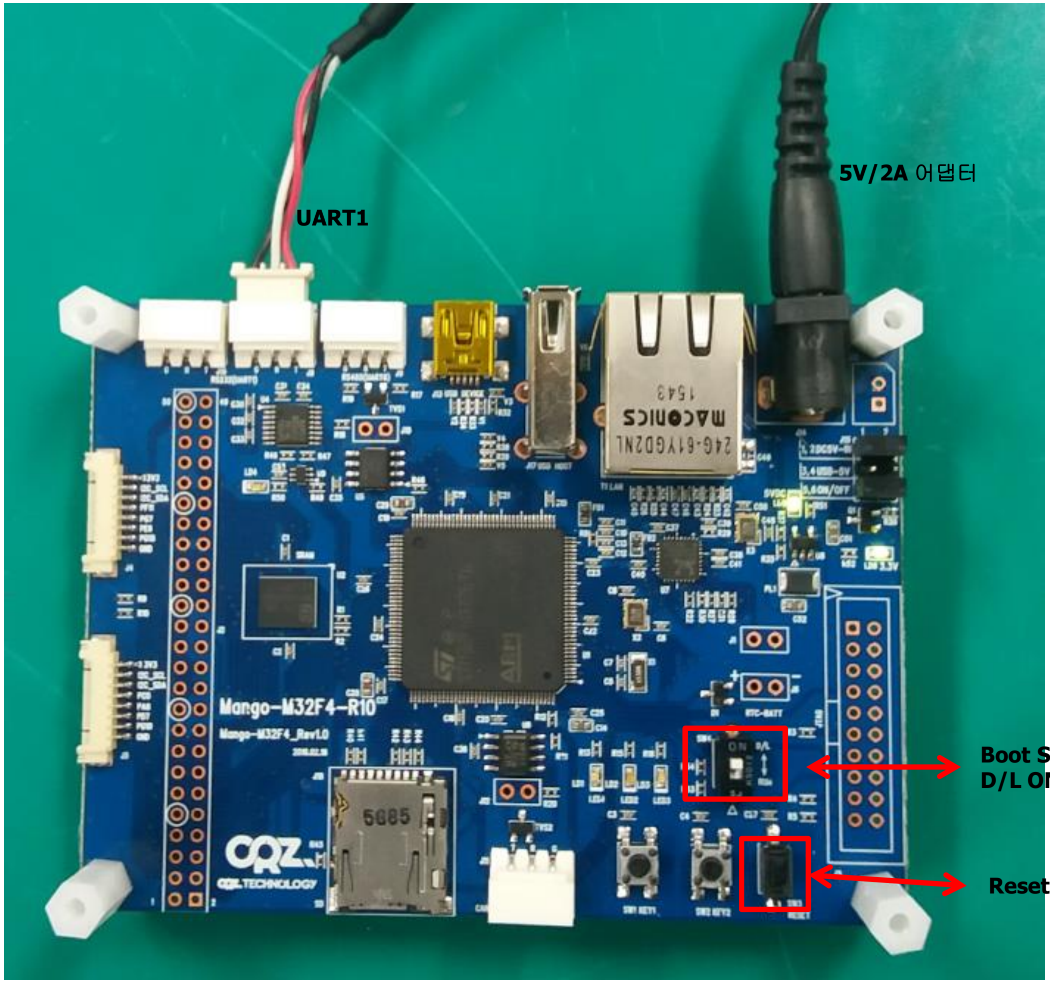
Build 결과물

- 빌드 결과물 이름은 [프로젝트 이름].bin이다.



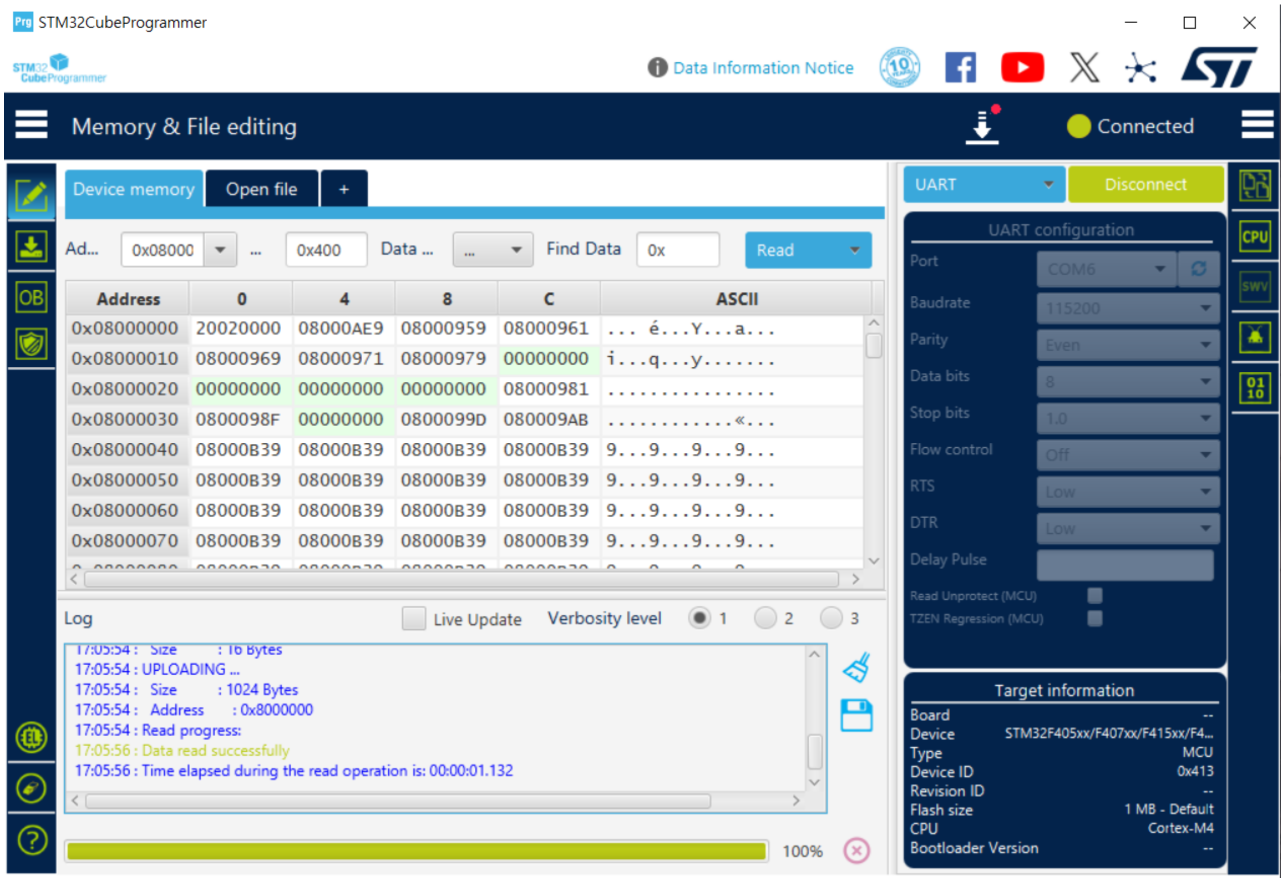
BIN 다운로드

- UART1과 연결되어 있는 터미널(Teraterm) 종료.
- Mango-M32F4를 다운로드 모드로 설정하고 SW3 리셋 키를 누른다.



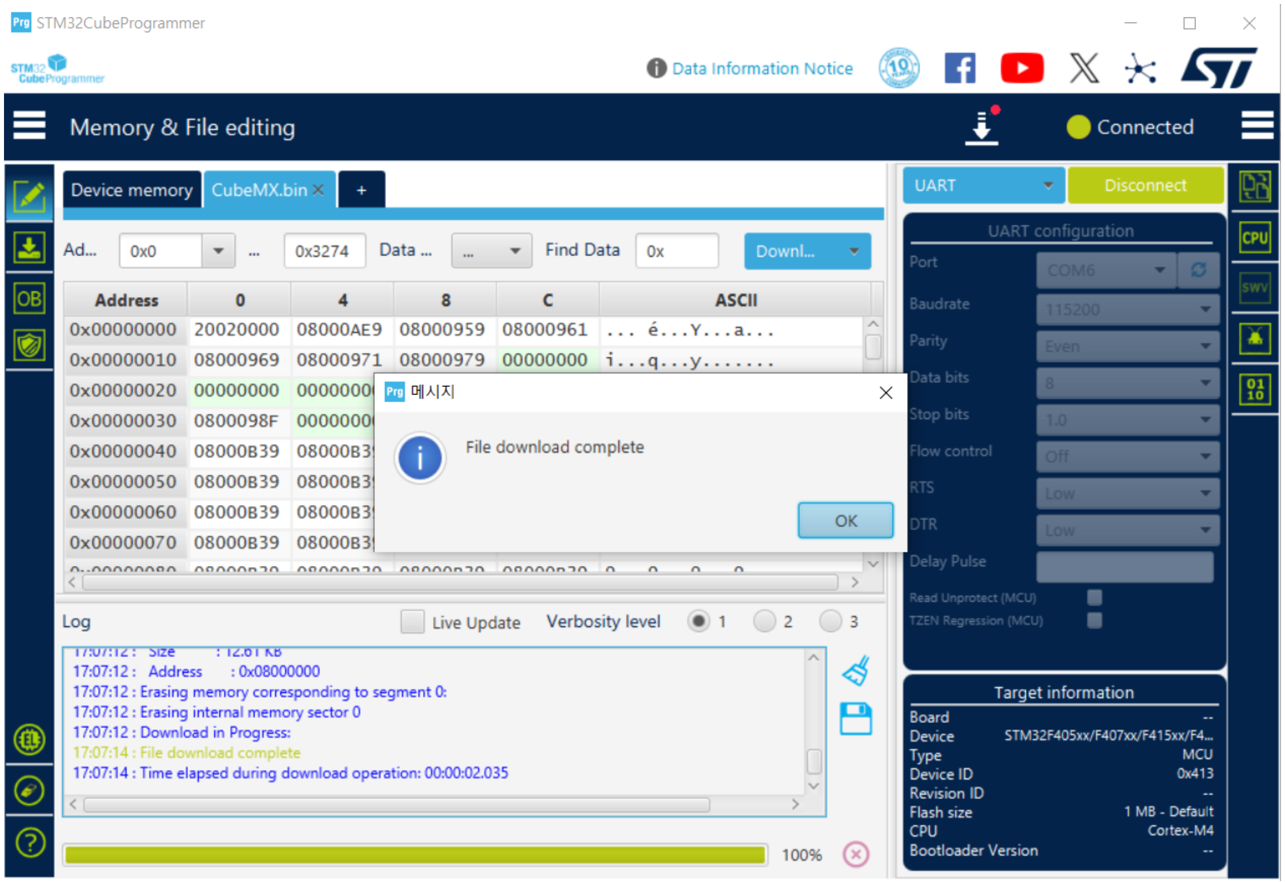
BIN 다운로드

- STM32CubeProgrammer를 실행한다.



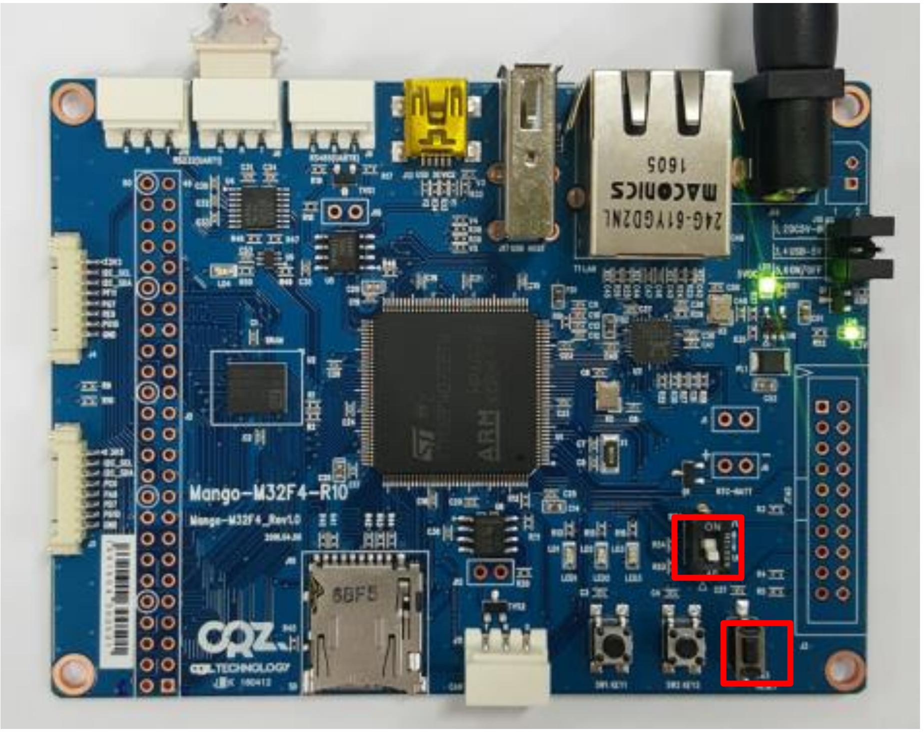
BIN 다운로드

- 빌드된 BIN 파일을 열어서 다운로드한다.



실행결과

- STM32CubeProgrammer의 COM Port를 연결해제하고 터미널(Teraterm)을 실행한다.
- Mango-M32F4를 실행모드로 설정하고 SW3 리셋 키를 누른다.



실행결과

- CX-HUMIDITY 센서 보드를 Mango-M32F4 보드의 J4 혹은 J5 에 8핀 커넥터를 이용하여 연결한다.

- 온도와 습도가 정상적으로 읽힘.